

How to Accelerate a PHP Slot Engine from 5,000 to 300,000+ Spins per Second and Save Millions on Infrastructure

Author: Andrii Starkov (Senior/Staff Software Engineer)

Project Repository: [GitHub Repository](#)

Professional Contacts: [LinkedIn Profile](#)

Table of Contents

1. **Introduction** (Test bench, base game, links to GitHub/LinkedIn)
2. **Analyzing Technical Debt in iGaming** (Calculation specifics, legacy, Monte Carlo as a Bottleneck)
3. **Architectural Decisions for Ultra Performance in PHP** (Killing OOP, flat arrays, Short-Circuit, micro-optimizations)
4. **Benchmark Results** (Comparing speeds: 5k vs 304k spins/sec, business profit)
5. **The Limit of Sanity: Dark Magic of Optimization** (Bitmasks, procedural chaos, precomputed win maps)
6. **Migrating to Go (Golang) in 3 Weeks** (Strong typing, slice optimization, millions of RPS)
7. **Conclusion** (PHP myths, "mechanical sympathy" for hardware)

1. Introduction

Hi! I am a Senior/Staff Software Engineer with over 11 years of experience in backend/frontend development, system design, and enterprise architecture. My profile is high-load systems with heavy legacy code, complex business logic, and refactoring ten-year-old monoliths "on the fly" without stopping production.

This article is the result of a deep analysis of architectural problems across hundreds of real games and a codebase that has accumulated over a decade. I completely rewrote the gaming core from scratch, moving all platform interaction logic outside of it. The engine became pure math.

Below is a detailed technical guide on how to force PHP to deliver the performance of commercial computing cores, why "beautiful" code kills the CPU, and how engineering decisions can save a business millions of dollars.

Environment and Test Bench Characteristics

All benchmarks and performance metrics discussed were captured on the exact same hardware:

- **OS:** Debian GNU/Linux 12 (bookworm) — Linux Kernel 6.1.0-44-amd64 (x86_64)
- **CPU:** 12th Gen Intel(R) Core(TM) i7-1260P (12 cores / 16 threads, up to 4.70 GHz)
- **RAM:** 32 GB
- **NVME:** Samsung SSD 990 PRO 1TB

- **Base Game:** For the purity of the experiment, a simple model was taken: a 3x3 game field, 7 symbols, 3 winning lines (horizontal from left to right). No free spins, respins, bonus rounds, or additional mechanics.

2. Analyzing Technical Debt in iGaming

Let's be honest: the end client does not care how beautiful the code is in your repository. The game spins, slots flash, the balance matches — for them, the system works. You can write messy code for 20 years, but for a business, internal architecture is either rocket fuel or a concrete block tied to its feet.

I had to face the codebase of a large iGaming project with a trail of hundreds of games, which was a textbook museum of technical debt. Here is just a small part of it:

- **Math in float:** All calculations were done in floating-point numbers, generating classic rounding and precision issues.
- **Versioning via massive switch-case sheets:** Used to support old game rules.
- **GodRepository:** All games and tools lived in a single place.
- **Monster files:** A single file could bloat to tens of thousands of lines of code, where 10 functions with wild naming patterns coexisted, doing exactly the same thing but "each in its own way."
- **Vacuum instead of tests and docs:** The codebase was passed down like the myths of ancient Greece — by word of mouth.

When a company has hundreds of games in its portfolio and tens of thousands of players spinning reels simultaneously, the server load becomes colossal. This is not a classic web CRUD. In a standard web application, the CPU mostly idles, waiting for a response from the database or network. To eliminate PHP bootstrap overhead per request there, we just use something like RoadRunner or Swoole. But in iGaming, the server is continuously occupied with pure, heavy computations of game logic for every single action of the player.

And this heavily heats the CPU, especially when running Monte Carlo simulations — calculating 10,000,000,000 (10 billion) spins for each RTP profile. This is a mandatory step for math validation, calculating volatility (Coefficient of Variation), and verifying RTP before sending software to strict certification labs (iTechLabs, eCOGRA).

If you write an iGaming PHP engine according to the academic canons of "clean OOP" (with objects for every symbol, line, and spin result), PHP suffocates on memory allocations and yields a miserable 100–200 spins per second. Tested from experience. Old legacy code written in the dinosaur era yielded roughly ~5,000 iterations per second due to heavy system functions (e.g., `array_merge`, `array_flip`), deeply nested structures causing Cache Misses, and constant disk I/O.

The Dry Math of Business Losses

If you need to run a baseline of 10 billion iterations at the old legacy engine's speed of 5,000 spins/sec, you get a catastrophic bottleneck:

$$10,000,000,000 / (5,000 \times 3,600) \approx 555.6 \text{ hours on a single core}$$

- On 10 cores in parallel — that's 55.5 hours of continuous work.
- On 20 cores — 27.7 hours.

And that is just one run. Dozens or hundreds of such cycles are required per game to adjust retention and volatility. As a result, Time-to-Market (TTM) stretches for weeks, and the company burns massive budgets renting multi-core cloud instances.

3. Architectural Decisions for Ultra Performance in PHP

Memory Destruction: Killing OOP in the Hot Path

The main enemy of extreme performance in PHP is memory allocation and Garbage Collection (GC). The loops generating game fields, searching for lines, and calculating payouts represent our Hot Path, executed billions of times. In the broader architecture, design patterns are vital. But inside the Hot Path, all academic beauty must die. The code here must be primitive, direct, and raw.

- **Objects go under the knife:** No objects inside the loop. Instantiating even a lightweight object like `new WinLine()` or `new SpinResult()` inside the Hot Path forces the CPU to spend hundreds of cycles on structure allocation and subsequent GC traversal. Only primitive data types and flat arrays belong here.
- **A conscious compromise:** Yes, arrays in PHP are ordered hash tables (zvals) and also allocate memory, and abandoning objects makes debugging harder. But this is a fundamental limitation of the language, and we consciously make this sacrifice. Outside the Hot Path, we write clean OOP, but before feeding data into the calculation loop, we strip them down to flat primitives.
- **No interfaces or runtime dynamics:** No polymorphic calls, "Strategy" patterns, or dynamic variables like `$$variable`. The PHP 8.5 JIT compiler cannot optimize code if data types and execution paths constantly shift at runtime. The CPU and JIT need a predictable flow control. Cleanliness inside the Hot Path must be absolute: if `calculateWin()` is designed to calculate wins, it does only that. No implicit logging, no side effects.
- **Cloning ban:** Forget the `clone` keyword, and especially deep cloning via heavy `serialize` / `unserialize`. All data preparation and transformations must finish before the first iteration of the loop triggers.

Flattening Matrices: The War Against Cache Misses

Two-dimensional arrays are the standard for representing a game grid: `Reels[X][Y]`. But accessing `$reels[$x][$y]` forces the CPU to endlessly jump across pointers, resulting in L1/L2 Cache Misses on large volumes. The CPU stalls waiting for data from slow RAM.

Solution: Glue all reels from left to right, top to bottom, into a single, flat 1D array. A 3x3 grid becomes an array with indices from 0 to 8.

$$position = rowID \times gameFieldWidth + reelID$$

Now, a winning line at the very top of a 3x3 grid is a simple index check of `0, 1, 2`. The data lies sequentially in memory and sits perfectly in the CPU's L1 cache.

Algorithmic Short-Circuit Optimization

When dealing with billions of operations, saving even 5% of resources translates into real infrastructure savings. We abandon universal calculators in favor of isolated methods or classes tailored to one specific

task. Algorithms are warmed up exactly once on the very first spin of the simulation, caching dimensions and pre-allocating memory structures that are simply overwritten later.

- **Checking lines instead of combinations:** If a game has 5 winning lines and 25 potential symbol combinations, we loop strictly over the 5 lines. Over a distance of 1 billion spins, this cuts out 20 billion internal loop iterations.
- **Entering from the second reel:** If combinations evaluate strictly left-to-right and require at least 3 symbols, we can start checking from the second reel (index 1). We look left to index 0. If the symbol on the left doesn't match and isn't a Wild, the line breaks immediately (`continue` / `break`), saving operations before even looking at the rest of the line.

```
// Concept of ultra-fast line check for 3 symbols (positions 0, 1, 2)
if ($flatReels[1] !== $flatReels[0]) {
    continue; // Line broken, 1st and 2nd symbols do not match
}
if ($flatReels[1] !== $flatReels[2]) {
    continue; // Line broken, 2nd and 3rd symbols do not match
}
// If reached here – a 3-symbol win is confirmed
```

Complex Wild logic is replaced by a flat configuration map — a **Lookup Table (LUT)**, turning deep logic checks into instantaneous array key lookups:

```
$wildMap = [
    'WILD_V1' => ['A' => true, 'B' => true, 'C' => true],
    'WILD_V2' => ['D' => true, 'E' => true],
];
```

Micro-Optimizations at the PHP Core Level

Slow Approach	Hot Path Optimized	Why It Works
<code>count(\$array)</code>	<code>\count(\$array)</code>	Calling from the global namespace (<code>\</code>) bypasses the current namespace lookup.
<code>array_merge()</code> , <code>array_flip()</code>	Manual iterative <code>for</code> loop	Built-in functions allocate new memory for every array. A simple <code>for</code> compiles perfectly under JIT.
Using <code>float</code> (0.05)	Using <code>int</code> (5)	All calculations use scaled x100 integers (cents/pence). CPUs compute integers significantly faster and with absolute precision.
<code>random_int()</code>	<code>\mt_rand()</code>	Cryptographic security is an overkill here. Over a large distance, the law of large numbers smooths out quirks, while speed scales multi-fold.

Critically Important: No xDebug enabled in the testing environment. Its overhead for tracking execution steps destroys raw computation performance entirely.

Separation of Environments: Production vs Emulation

In **Production**, you need a secure, verifiable, and certified RNG based on cryptographic primitives (e.g., `HMAC-SHA256`). But this is a massive speed killer. In **Emulation** mode (Monte Carlo), this logic is completely swapped for inline `\mt_rand`.

Emulation also mandates **Zero I/O**. Any log writing, file system interaction, or even a basic `echo` down to the console drops performance to the floor. Data processing runs purely in batches, writing state summaries only at the end.

4. Benchmark Results

With the new core architecture running on PHP 8.5 with JIT enabled, performance metrics for our 3x3 game field stabilized at the following marks:

Metric	Value
Iterations	10 000 000
Iterations per Second	304 853.12
Total Execution time	00:00:32.802

Result: 10 million spins are processed in just 32 seconds on a single core. Clean speed exceeds 304,000 spins/sec — a 60x (6,000%) speedup over the baseline legacy setup. Math verification cycles dropped from weeks to minutes, multiplying team output and slashing cloud compute infrastructure costs.

5. The Limit of Sanity: Dark Magic of Optimization

Can you push PHP further? Yes, but every step past this point breaks maintainability and clean design:

- **Bitmasks:** Processing line states and symbol matches via bitwise operations can yield an extra 25% to 50% speed boost, but renders the code unreadable.
- **Total Procedural Monolith:** Collapsing classes into a single monolithic file containing raw arrays and global scope functions.
- **Global State Store:** Bypassing data generation by interacting with a single, pre-allocated memory layout at the application root level.
- **Precomputed Win Combo Maps:** Mapping all possible layouts to outcomes on startup (e.g., `[A_A_A] => pay_multiplier`). It consumes vast amounts of RAM, but turns calculation into an `isset()` lookup. Useful for emulation, useless for Production HTTP cycles.
- **SpIFixedArray:** A common myth is that it always improves performance. For flat arrays with simple integer indices, it introduces overhead in PHP 8.5 compared to standard optimized packed arrays.

6. Migrating to Go (Golang) in 3 Weeks

To eliminate the remaining architectural bottlenecks inherent to the PHP runtime, I ported the entire core logic to Go (Golang). Interestingly, I had never written in Go before; my primary experience was in PHP and JS. The port took roughly 2–3 weeks of part-time work, treating AI engines not as code generators, but as highly specialized, strict search filters to answer thousands of explicit questions on memory layouts and syntax.

The Key Takeaway: PHP vs Go

In PHP, the greatest vulnerability remains the dynamic nature of packed arrays — a minor index shift or type omission surfaces only deep into simulations. Go halts this at compile time:

- **Static Typing as a Shield:** The compiler explicitly prevents runtime type chaos (e.g., trying to pass a `uint8` where an explicit `SymbolID` type is required).
- **Performance on Go:** Applying identical principles — Data Locality, fixed `[256]DataType` lookup tables, pre-allocated slice capacities, and ensuring calculations execute strictly on the CPU stack without heap allocations (verified via Escape Analysis) — yielded ****millions of spins per second on a single core.****

7. Conclusion

The slow speed of PHP under computational loads is a myth perpetuated by those who do not understand its internals. Grasping how the engine handles memory layouts (zvals), how JIT treats flat execution paths, and how CPUs leverage caching enables writing code with true ****Mechanical Sympathy**** for the hardware.

The code loses its conventional "web beauty," but gains genuine engineering elegance, slashing infrastructure expenses and accelerating development feedback loops without sacrificing production quality.